Task-12:- Technical Documentation(Monitoring Dashboards and Alerts for Bookstore API: Tracking Error Rates and HTTP Status Code Trends)

1. Introduction

1.1 Purpose

This document provides a detailed technical explanation of implementing monitoring dashboards and alerting for the Bookstore API deployed on Amazon Web Services.

The monitoring system helps track:

- Total API request count
- HTTP response status codes
- Error rates
- API performance trends

The monitoring dashboard enables the operations team to quickly identify failures and understand API behavior.

Intended Audience

- CTO
- DevOps Engineers
- Cloud Engineers
- Operations Team
- Developers

1.2 Scope

This document focuses on the following monitoring implementation:

- API observability using **New Relic APM**
- HTTP status code monitoring
- Error rate visualization
- Dashboard creation for operational monitoring
- Alert configuration for API failures

Out of Scope

- Kubernetes monitoring
- Distributed tracing implementation
- Advanced log analytics
- Enterprise-scale observability architecture

---

2. Overview of the Monitoring Project

2.1 Definition

API monitoring is the process of tracking the availability, performance, and response behavior of an API to ensure it is functioning correctly.

Monitoring helps detect:

- API failures
- Increasing error rates
- Abnormal traffic patterns
- Service outages

2.2 Key Principles – AWS Well-Architected Framework

1 Operational Excellence

Monitoring dashboards provide visibility into API behavior and system health.

2 Security

Monitoring ensures abnormal traffic or suspicious API activity is detected quickly.

3 Reliability

Alerts notify engineers when API failures occur.

4 Performance Efficiency

Monitoring response trends helps detect performance bottlenecks.

5 Cost Optimization

Early detection of errors prevents unnecessary infrastructure scaling.

6 Sustainability

Efficient monitoring ensures optimal system resource utilization.

2.3 Goals and Objectives

The primary objectives of the monitoring implementation are:

- Track API request volume
- Monitor HTTP response codes
- Detect rising error rates
- Visualize API response patterns
- Configure alerts for abnormal behavior

2.4 Constraints and Assumptions

Constraints

- Limited implementation time (3 hours)
- Basic observability setup
- Monitoring limited to API metrics

Assumptions

- Bookstore API already deployed
- New Relic agent installed on application servers
- Internet connectivity available
- API traffic generated during testing

---

3. Architecture and Infrastructure

3.1 Monitoring Architecture

Components used:

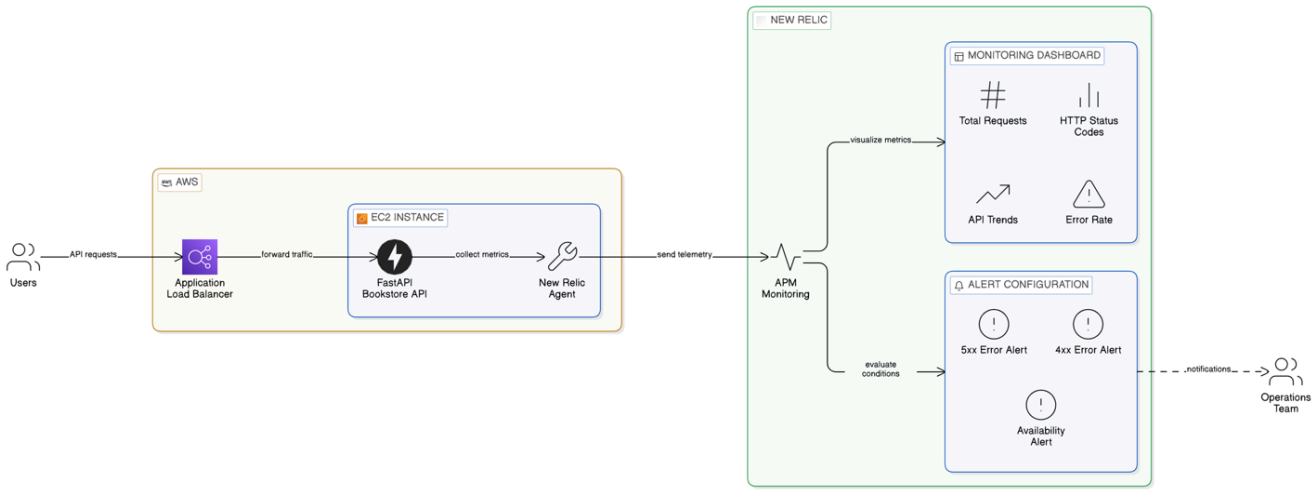
Component	Description
Amazon EC2	Hosts FastAPI application
AWS Application Load Balancer	Distributes traffic
New Relic	API monitoring and dashboards

3.2 Request Monitoring Flow

```
User Request
     |
     ▼
Application Load Balancer
     |
     ▼
FastAPI Application Server
     |
     ▼
New Relic Agent
     |
     ▼
New Relic Monitoring Dashboard
```

The monitoring platform collects metrics such as:

- Request count
- HTTP response codes
- Error rates
- API response trends



4. Dashboard Design

4.1 API Monitoring Dashboard

The monitoring dashboard provides visibility into API behavior using different visualizations.

Metrics Monitored

- Total Request Count
- Successful Responses (2xx)
- Client Errors (4xx)
- Server Errors (5xx)
- Overall Error Rate
- Request Trend Over Time

Install New Relic Agent

On EC2 instance:

```
pip install newrelic
```

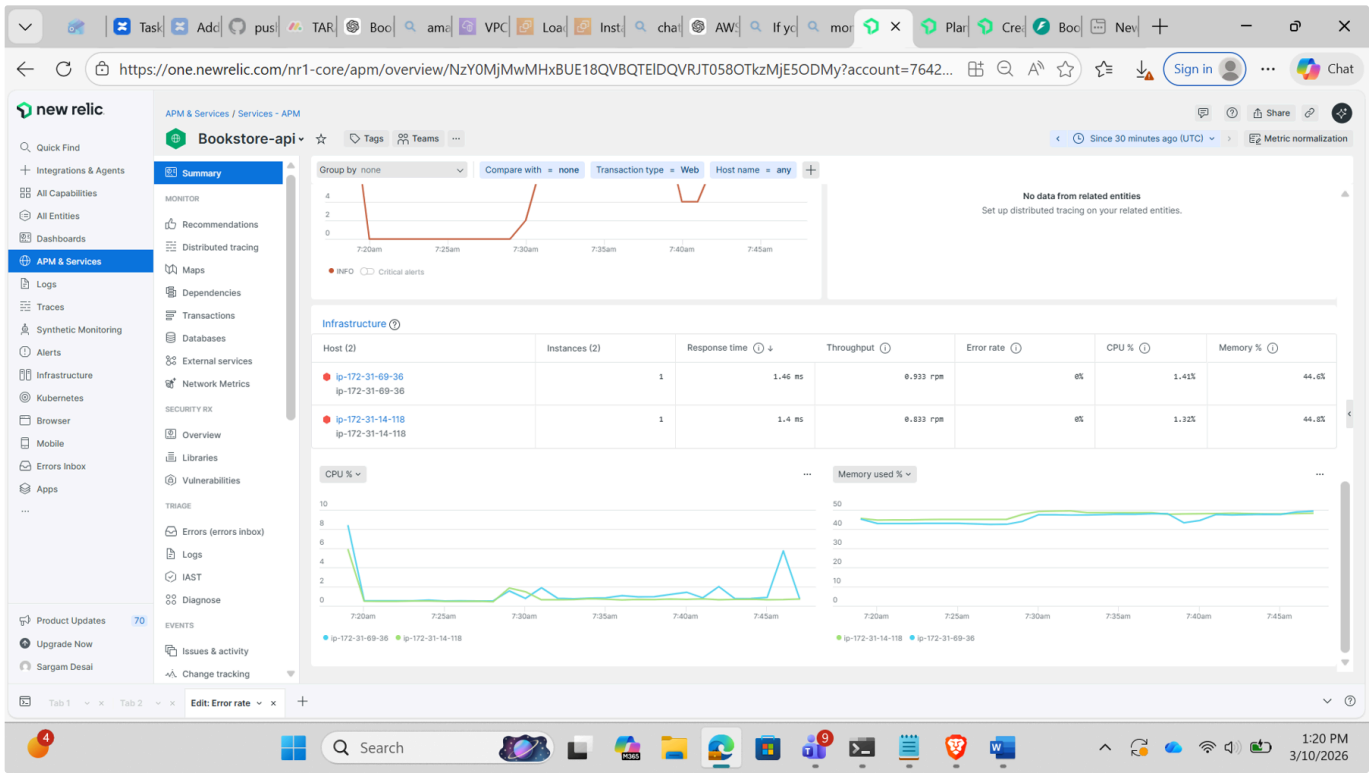
Initialize configuration:

```
newrelic-admin generate-config <license-key> newrelic.ini
```

Start application with monitoring:

```
newrelic-admin run-program uvicorn main:app --host 0.0.0.0 --port 8000
```

```
(venv) ubuntu@ip-172-31-69-36:~/bookstore-api/bookstore$ export MONGO_URI="mongodb://172.31.1.66:27017/bookstore"
(venv) ubuntu@ip-172-31-69-36:~/bookstore-api/bookstore$ NEW_RELIC_CONFIG_FILE=newrelic.ini newrelic-admin run-program uvicorn main:app --ho
st 0.0.0.0 --port 8000
INFO:     Started server process [71072]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
2026-03-10 07:42:00,575 (71072/MainThread) newrelic.core.agent INFO - New Relic Python Agent (11.5.0)
INFO:     172.31.79.22:3558 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.31.3.22:48432 - "GET /docs HTTP/1.1" 200 OK
2026-03-10 07:42:01,124 (71072/NR-Activate-Session/Bookstore-api) newrelic.core.agent_protocol INFO - Reporting to: https://rpm.newrelic.com
/accounts/7642300/applications/993219832
INFO:     172.31.79.22:21222 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.31.3.22:10502 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.31.79.22:16048 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.31.3.22:13948 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.31.79.22:44530 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.31.3.22:33204 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.31.79.22:25268 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.31.3.22:49036 - "GET /docs HTTP/1.1" 200 OK
```



4.2 Dashboard Widgets

1 Total Request Count

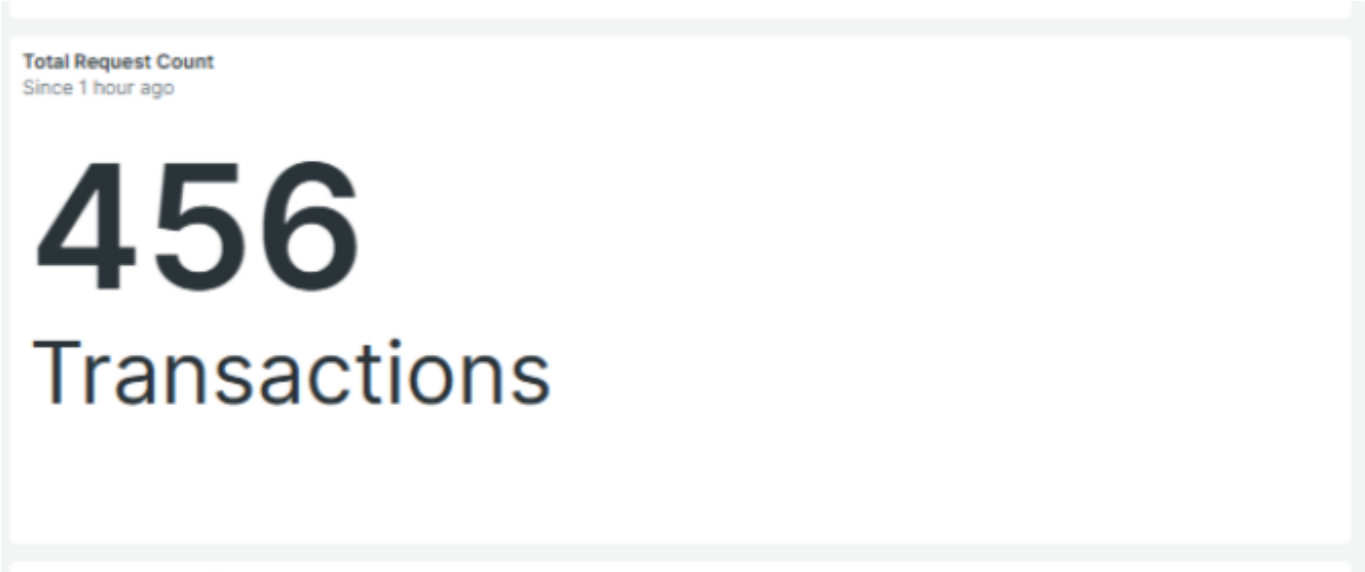
Displays the number of incoming API requests.

Visualization:

- Billboard

Example Query

```
SELECT count(*) FROM Transaction
```



---

2 HTTP Status Code Distribution

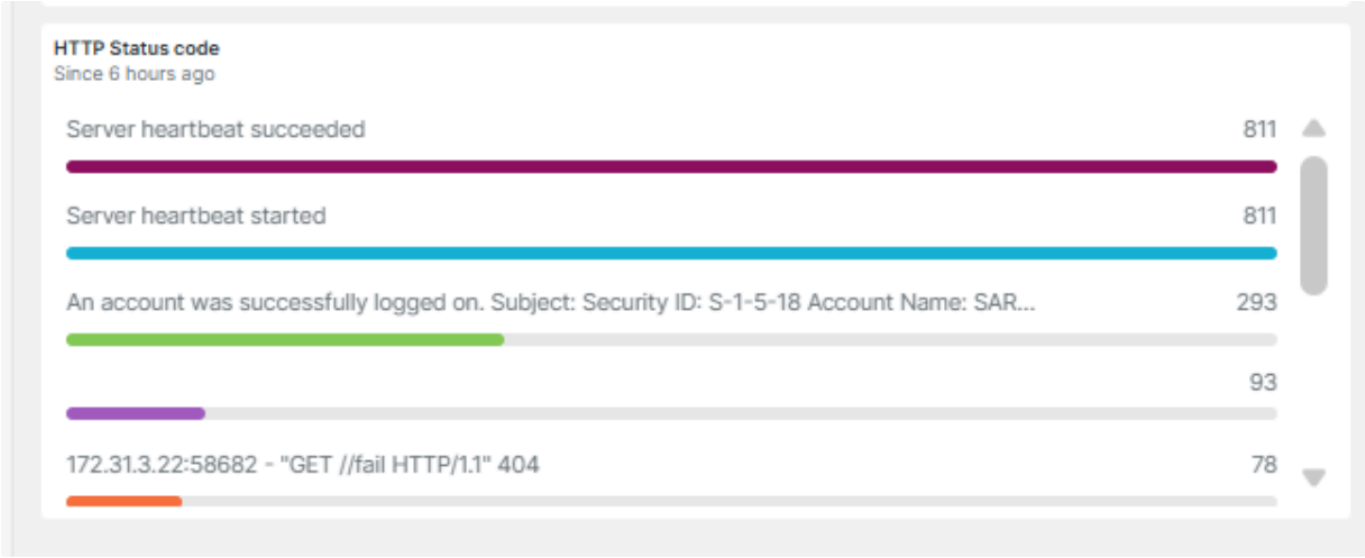
Shows response distribution.

Visualization:

- Bar chart

Example Query

```
SELECT count(*) FROM Transaction FACET httpResponseCode
```



---

3 API Response Trend

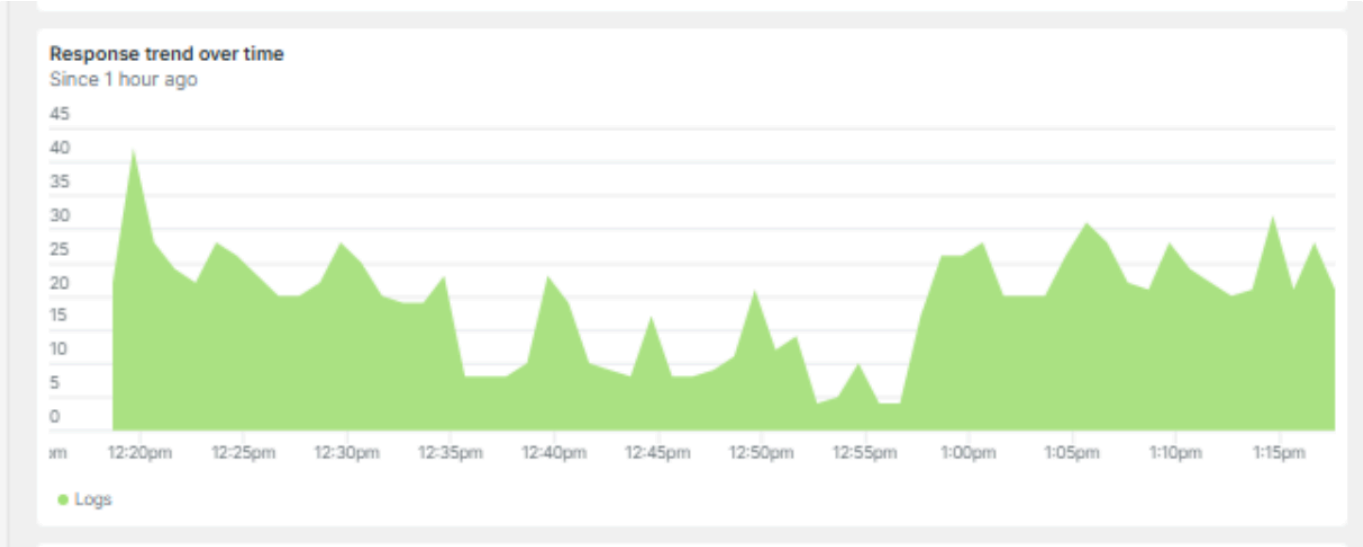
Displays request volume over time.

Visualization:

- Area chart

Example Query

```
SELECT count(*) FROM Transaction TIMESERIES
```



---

4 Error Rate Monitoring

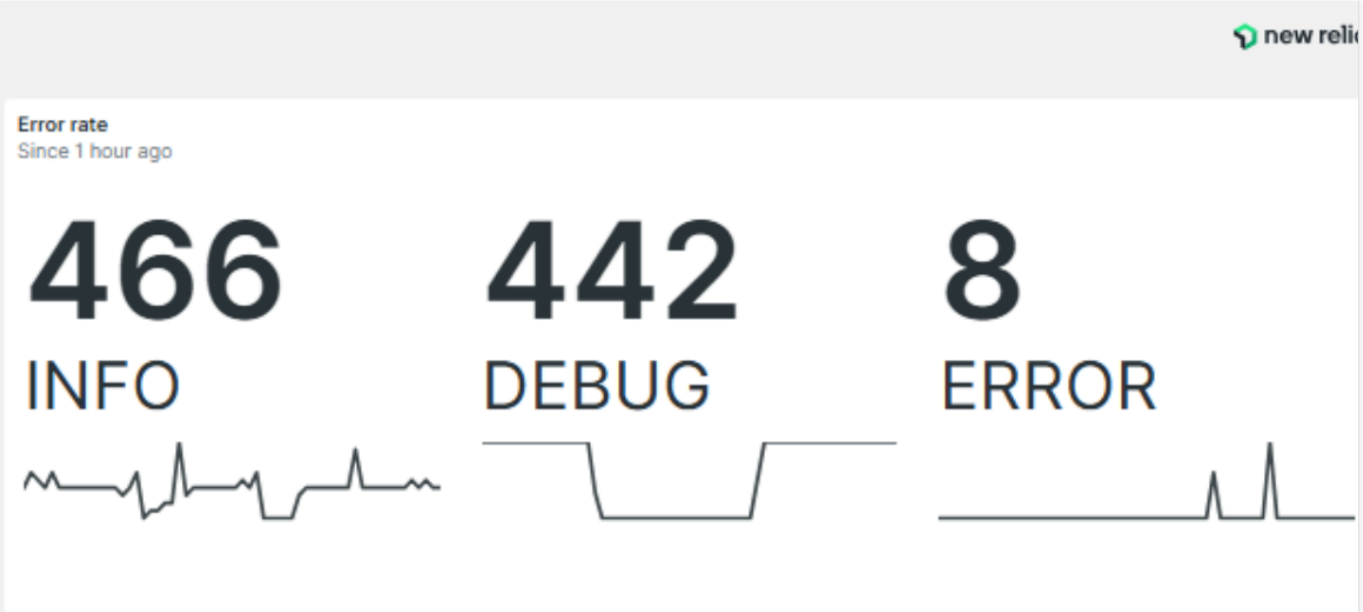
Displays percentage of failed requests.

Visualization:

- Billboard

Example Query

```
SELECT percentage(count(*), WHERE error IS true) FROM Transaction TIMESERIES
```



---

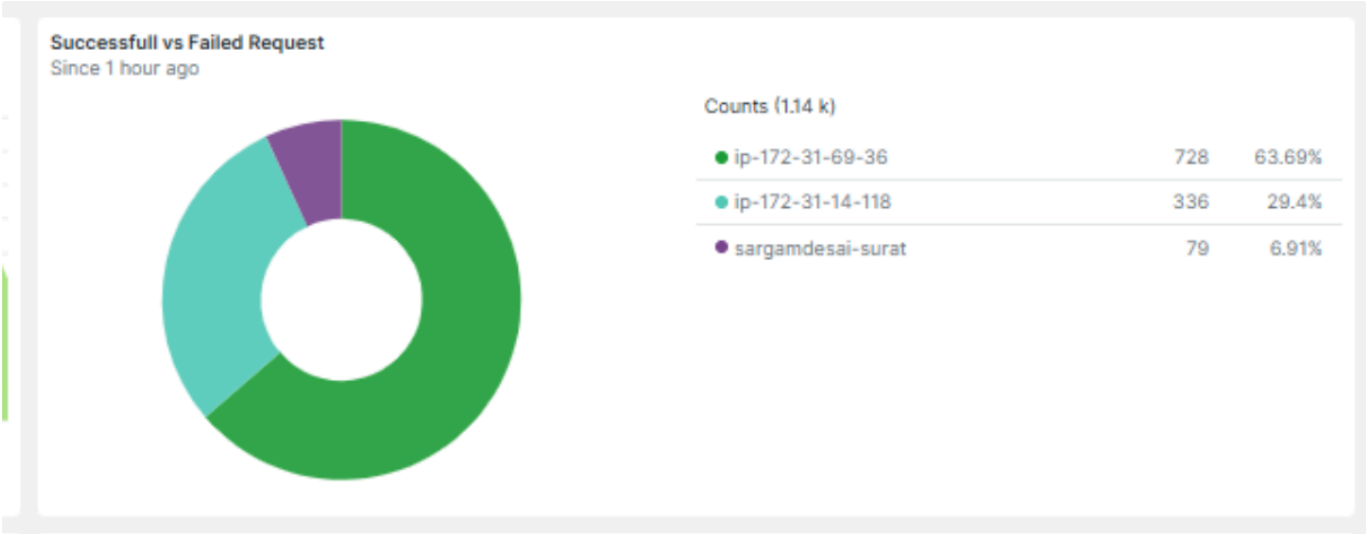
5 Success vs Failure Comparison

Visualization:

- Stacked bar chart

Example Query

```
SELECT count(*)
FROM Transaction
FACET error
```



5. Error Rate Monitoring

Error rate is a key indicator of API health.

Monitored Conditions

- Rising server errors
- Increased client errors
- Sudden drop in successful responses
- Abnormal request spikes

Change main.py file by import logging:

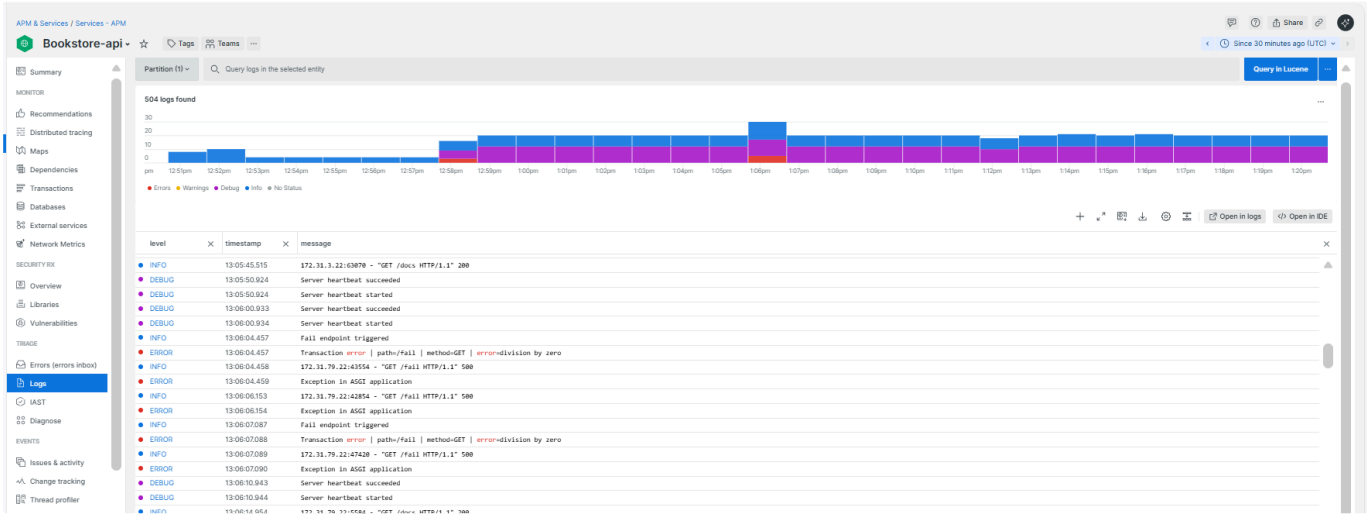
```
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from routers import book
import logging
# Logging configuration
logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s - %(levelname)s - %(message)s"
)
logger = logging.getLogger("bookstore-api")
app = FastAPI(title="BookStore API")
origins = [
    "localhost:3000"
]
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
app.include_router(book.router)
# Global exception handler (important for New Relic logs)
@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    logger.error(
        "Transaction error | path=%s | method=%s | error=%s",
        request.url.path,
        request.method,
        str(exc)
    )
    return JSONResponse(
        status_code=500,
        content={"detail": "Internal Server Error"}
    )
# Endpoint to generate test error
@app.get("/fail")
def fail():
    logger.info("Fail endpoint triggered")
    1/0

```

```
ue}
2026-03-10 10:12:07,373 (81739/MainThread) newrelic.core.agent INFO - New Relic Python Agent (11.5.0)
INFO:     172.31.79.22:15078 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.31.3.22:50650 - "GET /docs HTTP/1.1" 200 OK
2026-03-10 10:12:07,994 (81739/NR-Activate-Session/Bookstore-api) newrelic.core.agent_protocol INFO - Reporting to: https://rpm.newrelic.com/accounts/7642300/applications/993219832
2026-03-10 10:12:09,893 - DEBUG - {"message": "Server heartbeat succeeded", "topologyId": {"$oid": "69afee659217e8750e7fbfe4"}, "driverConnectionId": 1, "serverConnectionId": 35, "serverHost": "172.31.1.66", "serverPort": 27017, "awaited":
true, "durationMS": 10003.079963003984, "reply": "{\"isWritablePrimary\": true, \"topologyVersion\": {\"processId\": {\"$oid\": \"69afb69368da29df10953651\"}}, \"maxBsonObjectSize\": 16777216, \"maxMessageSizeBytes\": 48000000, \"maxWriteBa
tchSize\": 100000, \"localTime\": {\"$date\": \"2026-03-10T10:12:09.893Z\"}, \"logicalSessionTimeoutMinutes\": 30, \"connectionId\": 35, \"maxWireVersion\": 25, \"ok\": 1.0}"}
2026-03-10 10:12:09,895 - DEBUG - {"message": "Server heartbeat started", "topologyId": {"$oid": "69afee659217e8750e7fbfe4"}, "driverConnectionId": 1, "serverConnectionId": 35, "serverHost": "172.31.1.66", "serverPort": 27017, "awaited": tr
ue}
2026-03-10 10:12:11,665 - INFO - Fail endpoint triggered
2026-03-10 10:12:11,666 - ERROR - Transaction error | path=/fail | method=GET | error=division by zero
INFO:     172.31.3.22:60034 - "GET /fail HTTP/1.1" 500 Internal Server Error
ERROR:    Exception in ASGI application
Traceback (most recent call last):
  File "/home/ubuntu/venv/lib/python3.12/site-packages/uvicorn/protocols/http/httptools_impl.py", line 409, in run_asgi
    result = await app(  # type: ignore[func-returns-value]
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ubuntu/venv/lib/python3.12/site-packages/uvicorn/middleware/proxy_headers.py", line 60, in __call__
    return await self.app(scope, receive, send)
```

```
(venv) ubuntu@ip-172-31-69-36:~/bookstore-api/bookstore$ export MONGO_URI="mongodb://172.31.1.66:27017/bookstore"
(venv) ubuntu@ip-172-31-69-36:~/bookstore-api/bookstore$ NEW_RELIC_CONFIG_FILE=newrelic.ini newrelic-admin run-program uvicorn main:app --host 0.0.0.0 --port 8000
INFO:     Started server process [78603]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
DEBUG:pymongo.topology:{"message": "Server heartbeat succeeded", "topologyId": {"$oid": "69afe6bd15226dc4f1351f95"}, "driverConnectionId": 1, "serverConnectionId": 19, "serverHost": "172.31.1.66", "serverPort": 27017, "awaited": true, "dura
tionMS": 10011.385000997993, "reply": "{\"isWritablePrimary\": true, \"topologyVersion\": {\"processId\": {\"$oid\": \"69afb69368da29df10953651\"}}, \"maxBsonObjectSize\": 16777216, \"maxMessageSizeBytes\": 48000000, \"maxWriteBatchSize\":
100000, \"localTime\": {\"$date\": \"2026-03-10T09:39:19.646Z\"}, \"logicalSessionTimeoutMinutes\": 30, \"connectionId\": 19, \"maxWireVersion\": 25, \"ok\": 1.0}"}
DEBUG:pymongo.topology:{"message": "Server heartbeat started", "topologyId": {"$oid": "69afe6bd15226dc4f1351f95"}, "driverConnectionId": 1, "serverConnectionId": 19, "serverHost": "172.31.1.66", "serverPort": 27017, "awaited": true}
DEBUG:pymongo.topology:{"message": "Server heartbeat succeeded", "topologyId": {"$oid": "69afe6bd15226dc4f1351f95"}, "driverConnectionId": 1, "serverConnectionId": 19, "serverHost": "172.31.1.66", "serverPort": 27017, "awaited": true, "dura
tionMS": 10008.130001995596, "reply": "{\"isWritablePrimary\": true, \"topologyVersion\": {\"processId\": {\"$oid\": \"69afb69368da29df10953651\"}}, \"maxBsonObjectSize\": 16777216, \"maxMessageSizeBytes\": 48000000, \"maxWriteBatchSize\":
100000, \"localTime\": {\"$date\": \"2026-03-10T09:39:29.656Z\"}, \"logicalSessionTimeoutMinutes\": 30, \"connectionId\": 19, \"maxWireVersion\": 25, \"ok\": 1.0}"}
DEBUG:pymongo.topology:{"message": "Server heartbeat started", "topologyId": {"$oid": "69afe6bd15226dc4f1351f95"}, "driverConnectionId": 1, "serverConnectionId": 19, "serverHost": "172.31.1.66", "serverPort": 27017, "awaited": true}
2026-03-10 09:39:35,923 (78603/MainThread) newrelic.core.agent INFO - New Relic Python Agent (11.5.0)
```

Logs in APM:

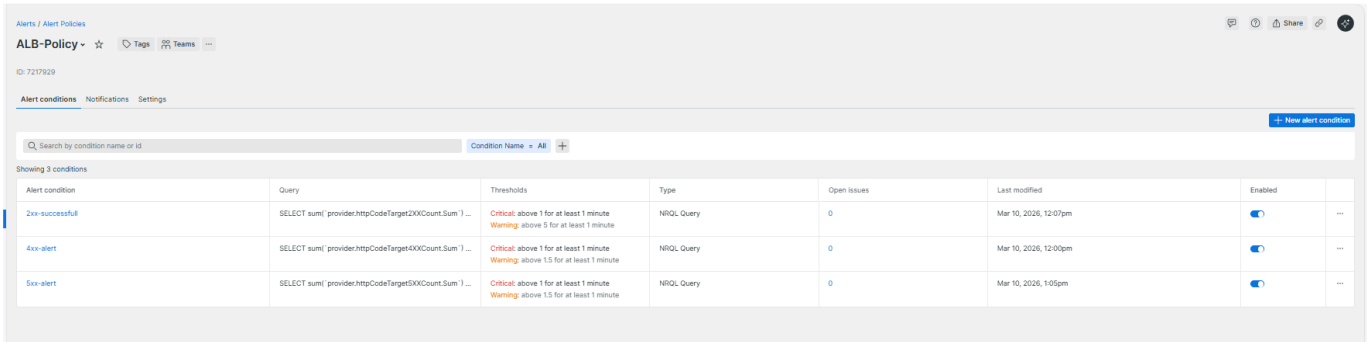


---

6. Alert Configuration

Alerts are configured to notify engineers when API behavior becomes abnormal.

Monitoring tool used: **New Relic Alerts**



6.1 High 5xx Error Rate Alert

Purpose:

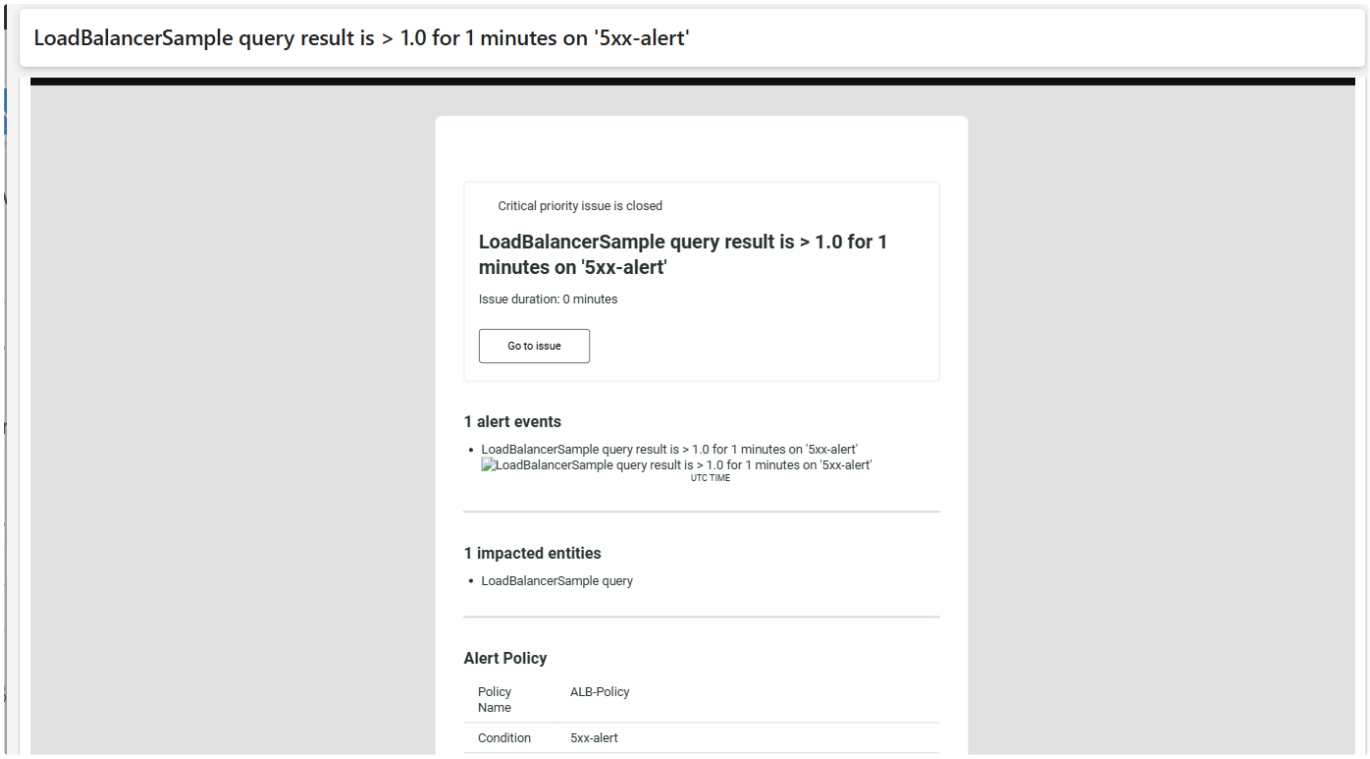
Detect server failures.

Condition:

```
5xx responses > 5% for 5 minutes
```

Impact:

Indicates backend application failure.



6.2 High 4xx Error Rate Alert

Purpose:

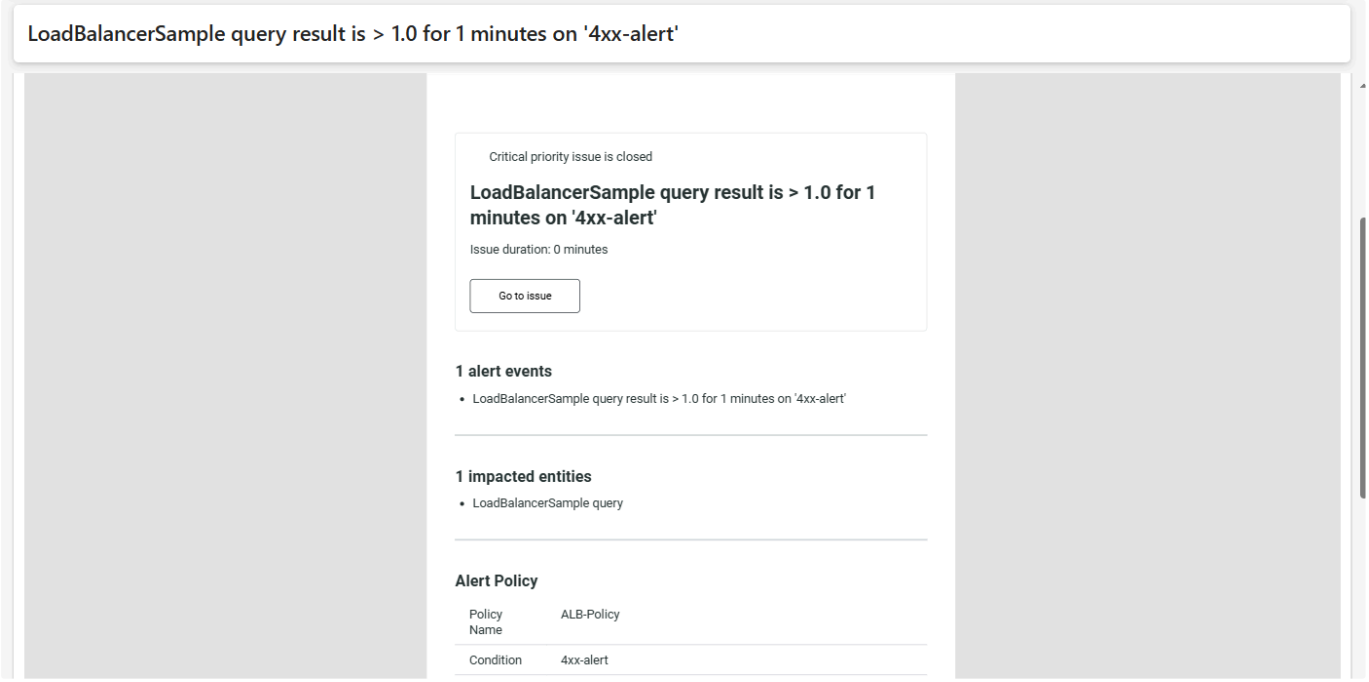
Detect excessive client errors.

Condition:

```
4xx responses > 20% of requests
```

Impact:

Indicates incorrect API usage or invalid requests.



6.3 Drop in Successful Responses

Purpose:

Detect API outage.

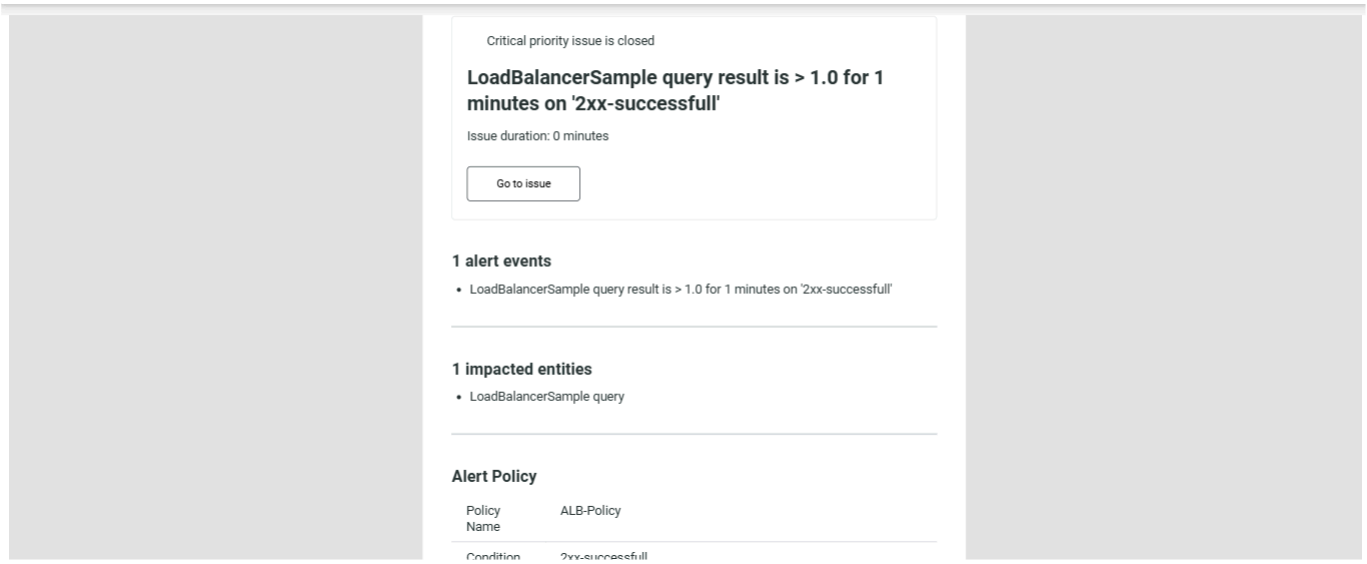
Condition:

```
2xx responses drop below threshold
```

Impact:

Possible API failure or infrastructure issue.

LoadBalancerSample query result is > 1.0 for 1 minutes on '2xx-successfull'



6.4 API Availability Alert

Purpose:

Detect when API becomes unreachable.

Condition:

```
No requests received for defined period
```

Impact:

Indicates API downtime.

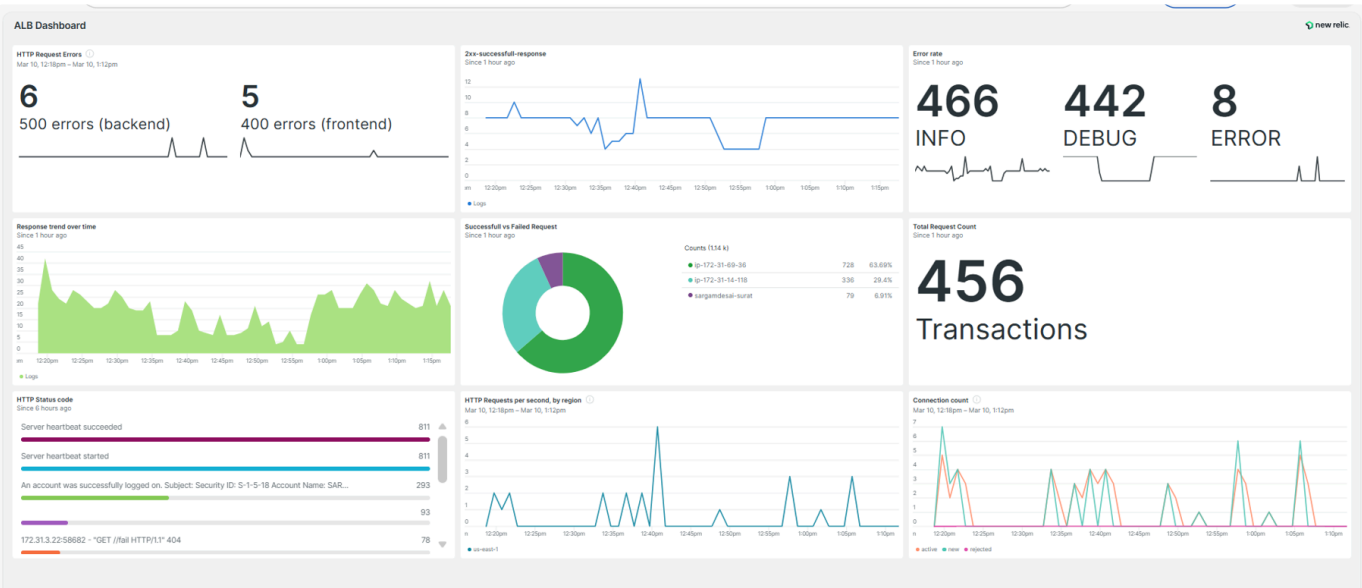
---

7. Incident Management

Incident workflow:

```
1 Monitoring detects anomaly
2 Alert triggered
3 DevOps engineer investigates logs
4 Root cause identified
5 Service restored
```

8. Dashboard



---

9. Implementation Timeline

Phase	Task	Duration
Phase 1	Monitoring tool setup	30 minutes
Phase 2	Dashboard creation	45 minutes
Phase 3	Alert configuration	30 minutes
Phase 4	Testing and validation	30 minutes

Total Implementation Time: **~2 hours**

---

10. Continuous Improvement

Future enhancements:

- Distributed tracing
- Centralized logging
- Prometheus + Grafana integration
- Auto-scaling alerts
- Latency monitoring

---

11. Expected Deliverables

1 Dashboard Screenshot

Monitoring dashboard showing:

- API request trends
- HTTP status code distribution
- Error rates

2 Alert Configuration Screenshot

Proof of configured alerts for:

- 5xx error rate

- 4xx spikes
- API availability

3 Confluence Documentation

Links provided for:

- Requirement analysis
- Technical documentation
- Error rate vs status code explanation

4 Demo Video

Short **2–3 minute video** explaining:

- Dashboard overview
- Error monitoring
- Alerts configuration

---

12. Conclusion

The Bookstore API monitoring dashboard was successfully implemented to improve observability and operational visibility.

Key Achievements

- API request monitoring
- HTTP status code tracking
- Error rate visualization
- Automated alerts for failures

The monitoring setup helps engineers quickly detect API issues and maintain service reliability.